How are IIIF Image API requests handled internally, pre-generated tiles vs dynamic rendering?

-> It is a dynamic Rendering and has a file-based cache

For now there are no pre-generated tiles (there is an idea to maybe do that to improve the cache). Currently when a viewer requests a “region-size-rotation-quality” combination, Djehuty does the following:

When a IIIF viewer first opens the image, it will request info.json before the images to get some hints about how to compute the images.This info.json tells the viewer information about the tiles size and scale factors to be computed from the image.

File wsgi.py
line 9523

```
def __iiif_image_context (self, metadata):
    """Returns a IIIF ImageService3 dict on success or None on failure."""
```

1. Opens the original image file from the storage)
2. Reads its width and height using pyvips
3. Computes the scale factors
4. Make the JSON dict
5. Returns it and after discards this info.json (no stored, process at the first view)

Them after it does the follow:

Step 1. Checks if there is already a file cache keyed with that MD5 combination (the name of the cached file is generated using file_uuid + region + size + rotation + quality + format). If the file with the specific requests exists in the cache Djehuty serves it immediately and it is done.

File wsgi.py
line 9754

```
# Serve from cache
# ----------------------------------------------------------------------
cache_key = self.db.cache.make_key ((f"{file_uuid}_{region}_{size}_"
                                     f"{mirror}_{rotation}_"
                                     f"{quality}_{image_format}"))
```

under the storage setup (it is configurable in the config file).

Step 2. If no cache exists yet, it fetches the file path from the database by using the file_uuid (if the image is on S3 storage it download the image again as a temp file) and if it is a PDF, add [dpi=300] so pyvips rasterizes it at 300 DPI.

After that, the code apply the requested transformations:

1. Crop: it cuts the requested region (x, y, width, height)
2. Resize: scale to the requested output
3. Mirror: flip horizontally
4. Rotate: rotate by the requested angle
5. Quality: convert to grayscale if gray was requested

File wsgi.py
line 9768

```
# Transform the input image to the output image
# ----------------------------------------------------------------------
output = None
try:
   output = pyvips.Image.new_temp_file(format=f".{image_format}")
   original.write(output)

   if s3_cached_file is not None:
       os.remove (s3_cached_file)

   output = output.crop (output_region["x"], output_region["y"],
                         output_region["w"], output_region["h"])
   output = output.thumbnail_image (output_size["w"],
                                    height = output_size["h"],
                                    size = "force")
   if mirror:
       output = output.fliphor()
   if rotation not in (0, 360):
       output = output.similarity (angle=rotation)
   if quality == "gray":
       output = output.colourspace (pyvips.enums.Interpretation.B_W)

   # The commonly used mimetype for TIFF is image/tiff.
   if image_format == "tif":
       image_format = "tiff"

   target = pyvips.Target.new_to_file (file_path)
   output.write_to_target (target, f".{image_format}")
   response = send_file (file_path, request.environ, f"image/{image_format}",
                     as_attachment=False, download_name=None)
   response.headers["Access-Control-Allow-Origin"] = "*"
```

Step 3. After the transformations Djehuty saves the result to the cache file (using the key id from the MD5 hash), then sends it as the HTTP response.

So the code is basically an image transformation pipeline: it receives the desired crop/size/rotation in the URL, transforms the original file, caches the result, and serves it and there is no pre-processing at upload time.

Supported source formats for the IIIF are: .jpg .png .pdf .tif .tiff .gif .webp. (PDFs are rasterized at 300 DPI).

2. Is manifest generation fully dynamic per request, or are manifests cached/precomputed?

The manifest is generated dynamic per request at the first time and after is cached per version. It is are built from the database on first request, then cached by (container_uuid, version_number).

The process is::

Step 1. First the code resolve the version:

In the URL if can cam can say latest, draft, or the version number like. We query the database to turn the latest/draft into the real version number

Step 2. We check the access. If the dataset is restricted or embargoed it will return 403 (Forbidden). Also a draft manifests require a logged-in user. (Because you can preview the dataset and the IIIF Manifest when it is still in draft).

Step 3. THere is a cache for the manifest as well. Cache key iiif_manifest_{container_uuid}_{version_number}. When the manifest is requested the code first checks if there is a cache and serves it. Otherwise it queries the database.

Step 4. If no cache the code builds the manifest by querying all files for the dataset
Then, for each file, call __iiif_image_context() (the same function that generates info.json).
If it succeeds (it means that the file is a supported format and it is readable), the file is included, otherwise the file is skipped. Then the it queries the dataset authors
and finally it calls format_iiif_manifest_record() to build the JSON.

Step 5. In the end it stores the cache for that manifest and returns it to the user.

Manifest JSON contains:

- label -> dataset title
- metadata -> contains authors (full names) + description of the dataset
- thumbnail -> the thumbnails for each image (pointing for the cached image)
- per file ->
    - label -> the filename
    - width / height -> actual image dimensions
    - the image URL (/full/max/0/default.jpg) plus the info.json context

Note: There are no cache invalidations . The cache is cleared whenever the dataset is updated. So a once requested published version is cached forever until the dataset changes (we can clean it manually in a dev process but there is no other way).

formatter.py
Line 742

```
def format_iiif_manifest_record (dataset, files, authors, version, base_url):
    """Record formatter for IIIF manifests."""
```

3. What mechanisms are used to link repository metadata (DOI, provenance) into IIIF Presentation structures?

I am not sure if I understood your question here.

What is included in the metadata today:

- Title -> goes into label
- Description -> goes into metadata array
- Authors -> goes into metadata array

What we do not have: (Everything else)

DOI, licenses, dates, funding, identifiers, organisations is not there